

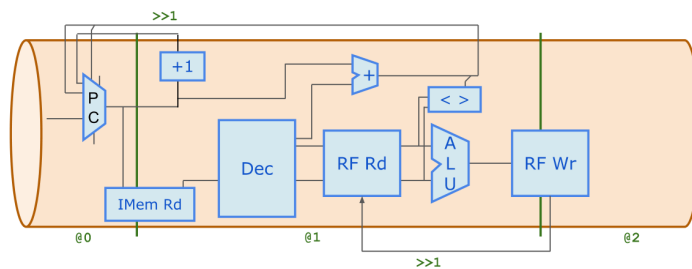
---

# RISC-V MYTH Workshop

## Day 5 — Pipelining the CPU

Lab Submission Report

### Pipelining Your RISC-V



**Workshop:** RISC-V MYTH (Microprocessor for You in Thirty Hours)

**Platform:** Makerchip IDE / TL-Verilog

**Provider:** Redwood EDA & VSD

**Day:** 5 of 5

---

|           |                                                     |           |
|-----------|-----------------------------------------------------|-----------|
| <b>1</b>  | <b>Overview of Day 5</b>                            | <b>2</b>  |
| <b>2</b>  | <b>Pipelining Concepts</b>                          | <b>3</b>  |
| 2.1       | Why Pipeline? . . . . .                             | 3         |
| 2.2       | TL-Verilog Timing Abstraction . . . . .             | 3         |
| <b>3</b>  | <b>Lab 1 — 3-Cycle Valid (\$valid)</b>              | <b>5</b>  |
| 3.1       | Theory . . . . .                                    | 5         |
| 3.2       | TL-Verilog Code . . . . .                           | 6         |
| <b>4</b>  | <b>Lab 2 — Register File (RF) Read/Write Hazard</b> | <b>7</b>  |
| 4.1       | Theory . . . . .                                    | 7         |
| 4.2       | TL-Verilog Code . . . . .                           | 8         |
| <b>5</b>  | <b>Lab 3 — Branches</b>                             | <b>9</b>  |
| 5.1       | Theory . . . . .                                    | 9         |
| 5.2       | TL-Verilog Code . . . . .                           | 9         |
| <b>6</b>  | <b>Lab 4 — Complete Instruction Decode</b>          | <b>11</b> |
| 6.1       | Theory . . . . .                                    | 11        |
| 6.2       | TL-Verilog Code . . . . .                           | 11        |
| <b>7</b>  | <b>Lab 5 — Complete ALU</b>                         | <b>13</b> |
| 7.1       | Theory . . . . .                                    | 13        |
| 7.2       | TL-Verilog Code . . . . .                           | 13        |
| <b>8</b>  | <b>Lab 6 — Loads and Data Memory</b>                | <b>15</b> |
| 8.1       | Theory . . . . .                                    | 15        |
| 8.2       | TL-Verilog Code . . . . .                           | 16        |
| <b>9</b>  | <b>Lab 7 — Jumps (JAL / JALR)</b>                   | <b>17</b> |
| 9.1       | Theory . . . . .                                    | 17        |
| 9.2       | TL-Verilog Code . . . . .                           | 17        |
| <b>10</b> | <b>Final CPU — Complete Pipelined RISC-V Core</b>   | <b>19</b> |
| 10.1      | Complete TL-Verilog Source . . . . .                | 20        |
| <b>11</b> | <b>Simulation Results</b>                           | <b>24</b> |
| <b>12</b> | <b>Summary</b>                                      | <b>26</b> |

## 1

Day 5 is the culminating session of the RISC-V MYTH workshop. The goal is to convert the single-cycle RISC-V CPU built on Day 4 into a **3-stage pipelined** design using TL-Verilog on the Makerchip IDE. The pipeline introduces *hazards* (data and control), which are resolved through carefully chosen forwarding and squashing strategies.

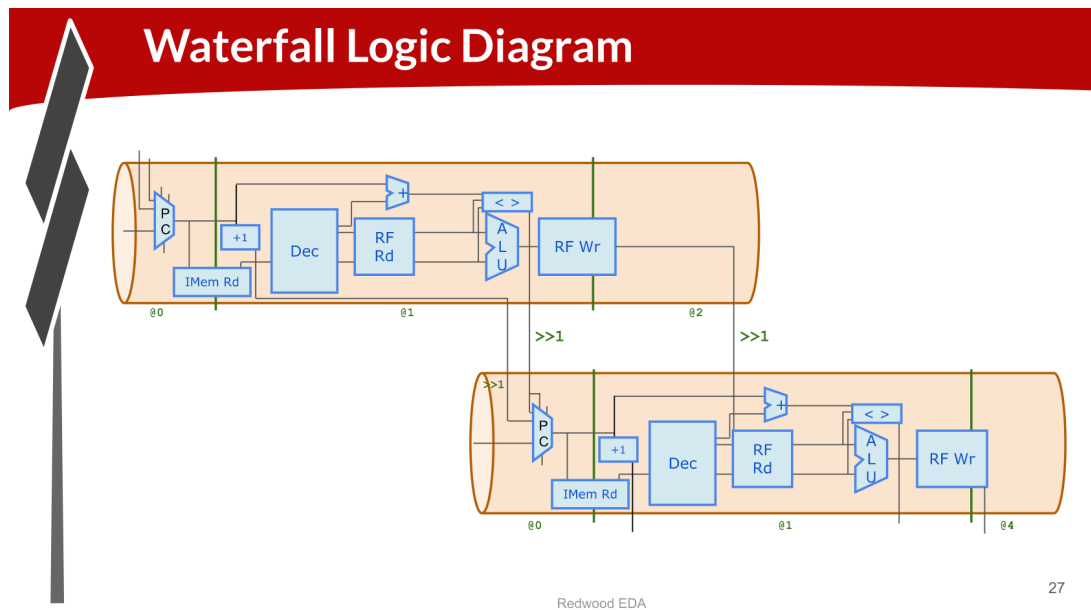


Figure 1: Day 5 agenda slide

The key milestones for the day are:

1. Introduce pipelining with a simple waterfall diagram and timing retiming.
2. Implement a 3-cycle `$valid` signal for instruction flow control.
3. Handle **register file (RF) read/write hazards** via bypassing.
4. Add **branch** support and redirect the PC.
5. Complete **instruction decode** for the full RV32I base set.
6. Complete the **ALU** for all RV32I instructions.
7. Integrate **load/store** instructions with Data Memory.
8. Connect the CPU to a **test program** and verify end-to-end.

## 2

## 2.1 Why Pipeline?

A single-cycle CPU must accommodate the *longest* instruction path in one clock period. Pipelining divides the datapath into stages so that multiple instructions overlap in time, raising throughput without reducing individual stage delay.

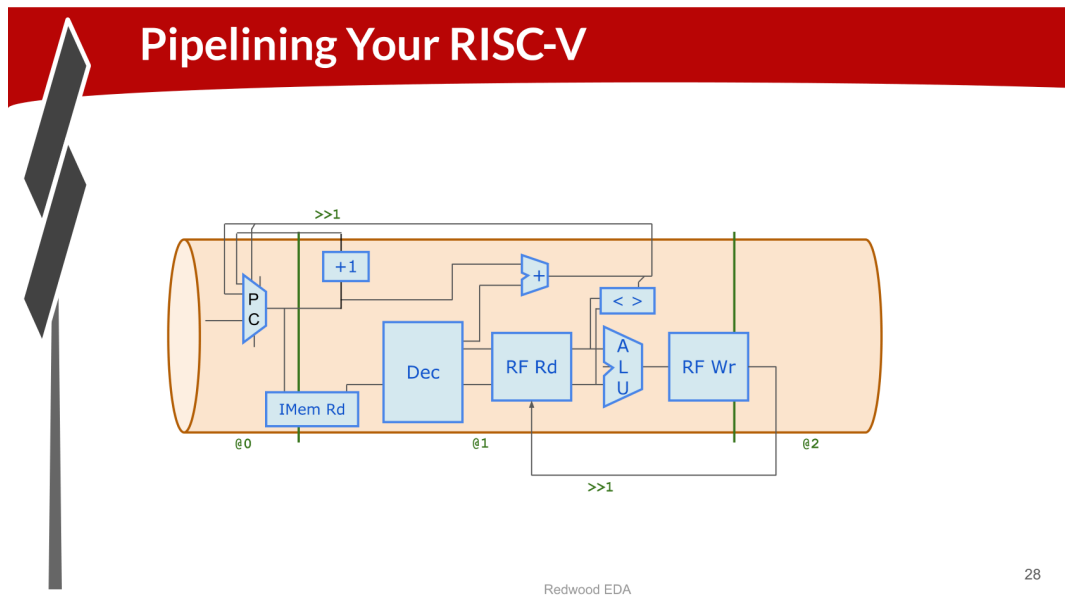
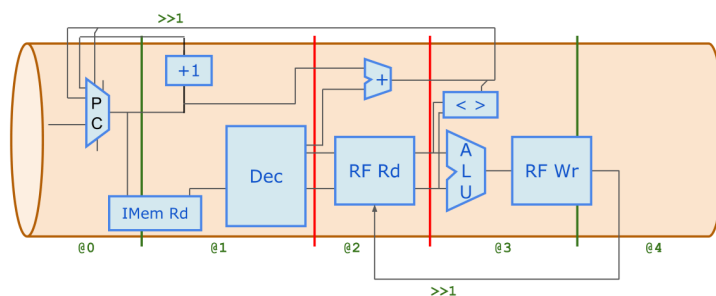


Figure 2: Waterfall diagram — three instructions flowing through the pipeline

## 2.2 TL-Verilog Timing Abstraction

TL-Verilog expresses pipeline stages with **timing retiming**. A signal referenced as `»N$sig` is the value of `$sig` from *N cycles ago*. This allows the designer to reason about logical intent without explicitly instantiating flip-flops between stages.

## Pipelining Your RISC-V

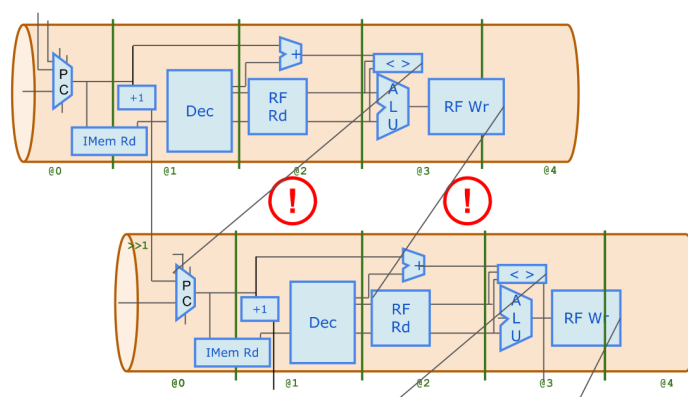


Redwood EDA

29

Figure 3: Timing / retiming concept in TL-Verilog

## Waterfall Logic Diagram



Redwood EDA

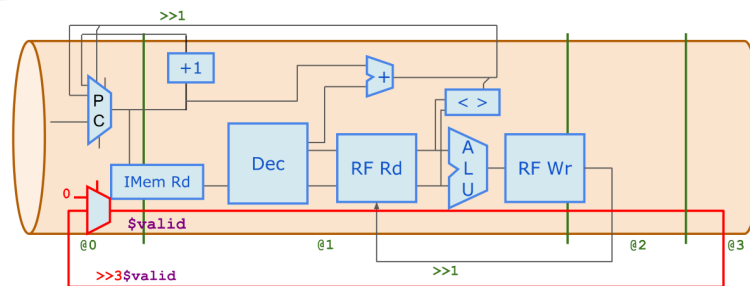
30

Figure 4: Pipeline stage boundaries and signal propagation

## 3

## Lab Objective

Introduce a `$valid` bit that controls whether each instruction in the pipeline is a “real” instruction or a pipeline bubble. This is needed because every branch or redirect injects 2 invalid cycles into the pipe.

Lab: 3-Cycle `$valid`

1. Create `$start` to provide first `$valid` pulse (reset last cycle, but not this cycle).
2. Create `$valid`: 0 during `$reset`, 1 for `$start`, `>>3$valid` o/w. Check simulation.



Figure 5: 3-cycle `$valid` pipeline diagram (slide 33)

## 3.1 Theory

When a branch is taken or the PC is redirected, the two instructions already fetched are *invalid*. Rather than stalling, this implementation marks them as invalid bubbles using `$valid`.

The key signals are:

- `$reset` — active while the CPU is being reset.
- `$start` — goes high the first cycle after reset deasserts (first valid fetch).
- `$valid` — 0 during reset, 1 on start, otherwise follows `>>3$valid` (the valid bit 3 cycles ago) to repeat every 3 cycles at start-up.

### 3.2 TL-Verilog Code

```
1 // -- 3-Cycle $valid -----
2 // $start: first cycle after reset deasserts
3 $start = $reset ? 1'b0 : (>>1$reset ? 1'b1 : 1'b0);
4
5 // $valid: 0 during reset, 1 for $start, >>3$valid otherwise
6 $valid = $reset ? 1'b0
7           : ($start ? 1'b1 : >>3$valid);
```

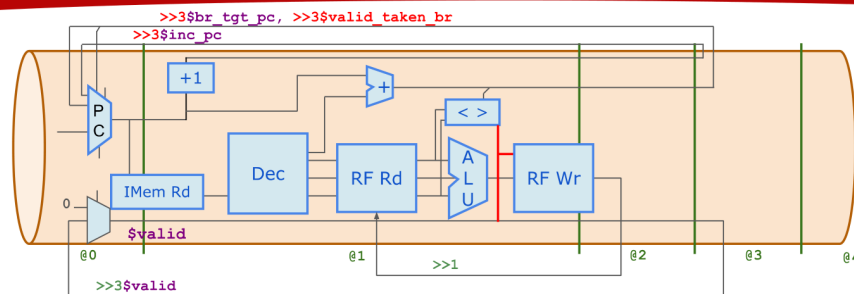
**Simulation check:** The waveform should show **\$reset** going low, then **\$start** pulsing high for exactly *one* cycle, followed by **\$valid** repeating with a pattern that produces a 1 every 3 cycles until the pipeline fills. The slide waveform (bottom of slide 33) confirms this pattern.



## Lab Objective

Detect and resolve **read-after-write** (RAW) hazards where an instruction reads a register that was written by one of the two immediately preceding instructions.

## Lab: 3-Cycle RISC-V



1. Avoid writing RF for invalid instructions.
2. Avoid redirecting PC for invalid (branch) instructions.  
Introduce: `$valid_taken_br = $valid && $taken_br`; and use it in PC mux.
3. Update inter-instruction dependency alignments (`>>3`).
4. Debug until passing. Confirm save.

Redwood EDA

36

Figure 6: RF hazard detection slide

## 4.1 Theory

In a 3-stage pipeline:

- Instruction at `@3` reads the register file at `@2`.
- The instruction that is *1 cycle older* (at `»1`) may still be writing to RF.
- The instruction that is *2 cycles older* (at `»2`) may also be in the write stage.

Therefore we need to forward (bypass) the result from those in-flight instructions rather than reading stale RF data.

## 4.2 TL-Verilog Code

```
1 // -- RF Read/Write Hazard Detection and Bypass -----
2 // Detect hazard: src register matches dest of recent writes
3 $src1_value[31:0] =
4     ((>>1$rf_wr_en && (>>1$rd == $rs1)) ? >>1$result :
5     (>>2$rf_wr_en && (>>2$rd == $rs1)) ? >>2$result :
6         $rf_rd_data1);
7
8 $src2_value[31:0] =
9     ((>>1$rf_wr_en && (>>1$rd == $rs2)) ? >>1$result :
10    (>>2$rf_wr_en && (>>2$rd == $rs2)) ? >>2$result :
11        $rf_rd_data2);
```

**Priority:** The most-recent instruction (»1) takes priority over the older one (»2). Both are checked before falling back to the register file read data.

### Lab Objective

Implement branch instructions (BEQ, BNE, BLT, BGE, BLTU, BGEU) and redirect the Program Counter when a branch is taken, invalidating the two mis-fetched instructions.

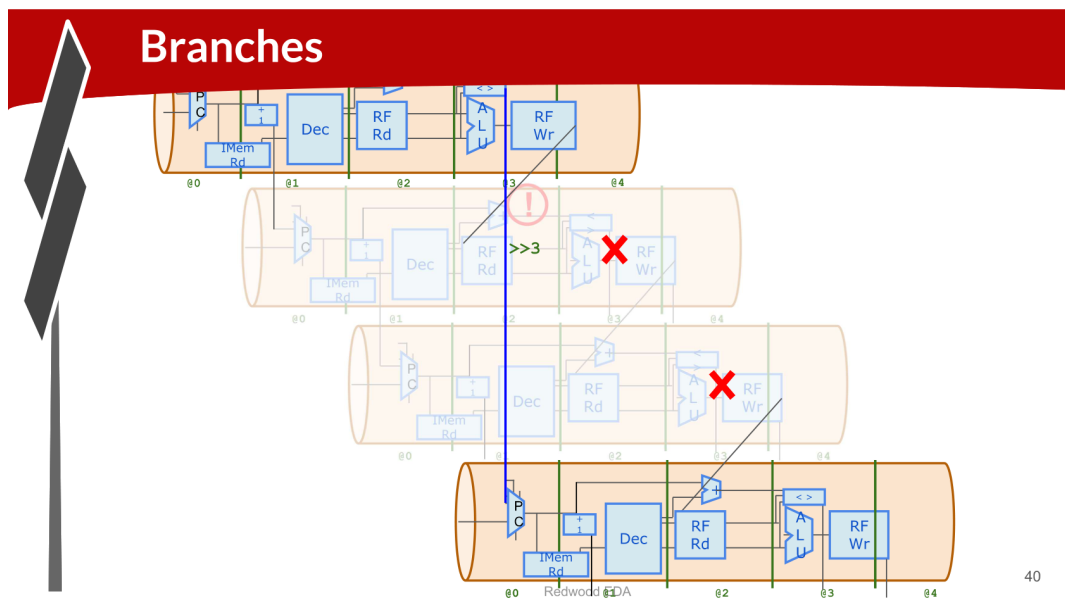


Figure 7: Branch pipeline diagram showing 2 squashed instructions

### 5.1 Theory

Branches are resolved in the ALU stage (@3). The PC must be corrected by feeding  $\gg 3 \$br\_tgt\_pc$  back to the PC stage, and the two instructions fetched in the interim must be squashed (marked invalid).

The branch target address is computed as  $\$pc + \$imm$  (sign-extended offset).

### 5.2 TL-Verilog Code

```

1 // -- Branch Logic -----
2 // Branch target
3 $br_tgt_pc[31:0] = $pc + $imm;
4
5 // Taken-branch condition per instruction type
6 $taken_branch =
7     $is_beq ? ($src1_value == $src2_value) :
8     $is_bne ? ($src1_value != $src2_value) :
```

```

9      $is_blt  ? (($src1_value < $src2_value) ^
10                ($src1_value[31] != $src2_value[31])):
11      $is_bge  ? (($src1_value >= $src2_value) ^
12                ($src1_value[31] != $src2_value[31])):
13      $is_bltu ? ($src1_value < $src2_value)          :
14      $is_bgeu ? ($src1_value >= $src2_value)          :
15                1'b0;
16
17  // PC redirect: use branch target 3 cycles ago if taken
18  $valid_taken_branch = $valid && $taken_branch;
19
20  // PC mux  redirect wins over PC+4
21  $pc[31:0] =
22      >>1$reset          ? 32'b0 :
23      >>3$valid_taken_branch ? >>3$br_tgt_pc :
24                          >>1$pc + 32'd4;
25
26  // Valid bit: squash the 2 cycles after a taken branch
27  $valid = $reset ? 1'b0
28           : ($start ? 1'b1
29              : (>>3$valid && !>>1$valid_taken_branch
30                && !>>2$valid_taken_branch));

```

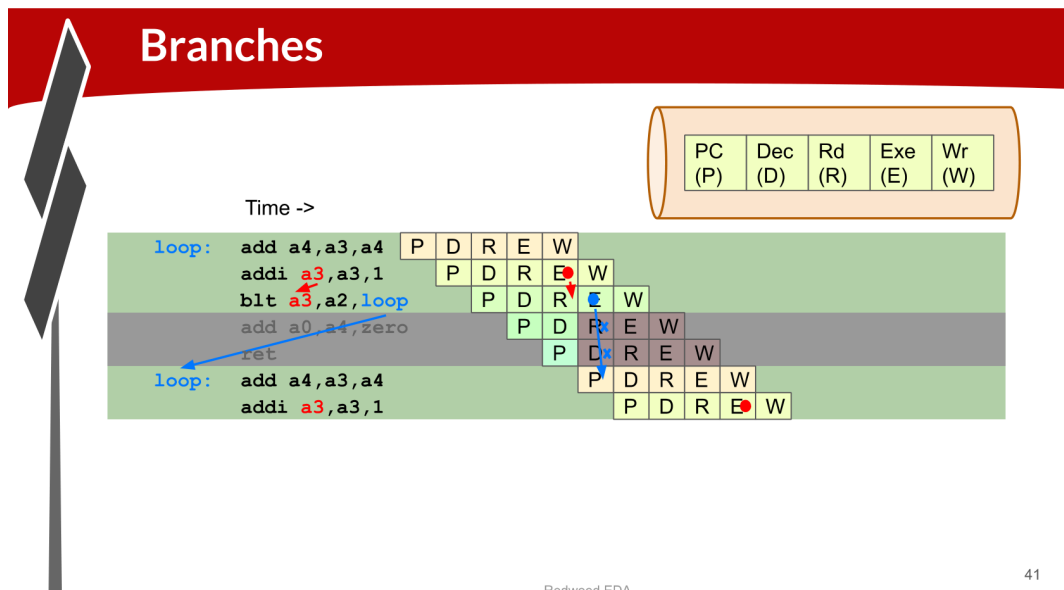


Figure 8: Lab: Branches — implementation slide

### Lab Objective

Decode all RV32I base instructions (except FENCE, ECALL, EBREAK). Generate a boolean flag for every instruction type, enabling the ALU and control logic to select the correct operation.

## Lab: Complete Instruction Decode

RV32I Base Instruction Set (except FENCE, ECALL, EBREAK):

| opcode  | func3   | func7[5] | func3 | opcode  |
|---------|---------|----------|-------|---------|
| 0110111 |         |          | 011   | 0010011 |
| 0010111 |         |          | 100   | 0010011 |
| 1101111 |         |          | 110   | 0010011 |
| 1100111 |         |          | 111   | 0010011 |
| 000     | 1100011 |          | 001   | 0010011 |
| 001     | 1100011 |          | 101   | 0010011 |
| 001     | 1100011 |          | 101   | 0010011 |
| 100     | 1100011 |          | 000   | 0110011 |
| 101     | 1100011 |          | 000   | 0110011 |
| 110     | 1100011 |          | 001   | 0110011 |
| 111     | 1100011 |          | 010   | 0110011 |
| 000     | 0000011 |          | 011   | 0110011 |
| 001     | 0000011 |          | 100   | 0110011 |
| 010     | 0000011 |          | 101   | 0110011 |
| 100     | 0000011 |          | 101   | 0110011 |
| 101     | 0000011 |          | 110   | 0110011 |
| 000     | 0100011 |          | 111   | 0110011 |
| 001     | 0100011 |          |       |         |
| 010     | 0100011 |          |       |         |
| 000     | 0010011 |          |       |         |
| 010     | 0010011 |          |       |         |
| 000     | 0010011 |          |       |         |
| 010     | 0010011 |          |       |         |

1. Complete remaining instrs, except loads (L\*).

```
$dec_bits[10:0] =
  {$func7[5], $func3, $opcode};
$iss_beq = $dec_bits ==?
  11'bx_000_1100011;
```

```
// Until instrs are implemented,
// quiet down the warnings.
`BOGUS_USE($iss_beq $iss_bne ...)
```

2. We'll treat all loads the same, so generate `$iss_load` based on opcode only.
3. Confirm save.

44

Redwood EDA

Figure 9: Complete instruction decode — RV32I table and code hints

### 6.1 Theory

The decode uses `dec_bits` formed by concatenating `$func7[5]`, `$func3`, and `$opcode` into an 11-bit vector. Each instruction is then identified by comparing against its encoding.

### 6.2 TL-Verilog Code

```
1 // -- Decode bits -----
```

```

2 $dec_bits[10:0] = {$funct7[5], $funct3, $opcode};
3
4 // -- R-type -----
5 $is_add  = $dec_bits ==? 11'b0_000_0110011;
6 $is_sub  = $dec_bits ==? 11'b1_000_0110011;
7 $is_sll  = $dec_bits ==? 11'b0_001_0110011;
8 $is_slt  = $dec_bits ==? 11'b0_010_0110011;
9 $is_sltu = $dec_bits ==? 11'b0_011_0110011;
10 $is_xor  = $dec_bits ==? 11'b0_100_0110011;
11 $is_srl  = $dec_bits ==? 11'b0_101_0110011;
12 $is_sra  = $dec_bits ==? 11'b1_101_0110011;
13 $is_or   = $dec_bits ==? 11'b0_110_0110011;
14 $is_and  = $dec_bits ==? 11'b0_111_0110011;
15
16 // -- I-type ALU -----
17 $is_addi = $dec_bits ==? 11'bx_000_0010011;
18 $is_slti = $dec_bits ==? 11'bx_010_0010011;
19 $is_sltiu = $dec_bits ==? 11'bx_011_0010011;
20 $is_xori = $dec_bits ==? 11'bx_100_0010011;
21 $is_ori  = $dec_bits ==? 11'bx_110_0010011;
22 $is_andi = $dec_bits ==? 11'bx_111_0010011;
23 $is_slli = $dec_bits ==? 11'b0_001_0010011;
24 $is_srli = $dec_bits ==? 11'b0_101_0010011;
25 $is_srai = $dec_bits ==? 11'b1_101_0010011;
26
27 // -- Branches -----
28 $is_beq  = $dec_bits ==? 11'bx_000_1100011;
29 $is_bne  = $dec_bits ==? 11'bx_001_1100011;
30 $is_blt  = $dec_bits ==? 11'bx_100_1100011;
31 $is_bge  = $dec_bits ==? 11'bx_101_1100011;
32 $is_bltu = $dec_bits ==? 11'bx_110_1100011;
33 $is_bgeu = $dec_bits ==? 11'bx_111_1100011;
34
35 // -- Load / Store -----
36 $is_load = $opcode == 7'b0000011; // all loads treated same
37 $is_sb   = $dec_bits ==? 11'bx_000_0100011;
38 $is_sh   = $dec_bits ==? 11'bx_001_0100011;
39 $is_sw   = $dec_bits ==? 11'bx_010_0100011;
40
41 // -- U-type / J-type -----
42 $is_lui  = $opcode == 7'b0110111;
43 $is_auipc = $opcode == 7'b0010111;
44 $is_jal  = $opcode == 7'b1101111;
45 $is_jalr = $dec_bits ==? 11'bx_000_1100111;
46
47 // Quiet down warnings during development
48 'BOGUS_USE($is_beq $is_bne $is_blt $is_bge $is_bltu $is_bgeu
49           $is_slt $is_sltu $is_slti $is_sltiu)

```

## Lab Objective

Implement the full Arithmetic-Logic Unit to produce **\$result** for every RV32I instruction. Some instructions (SLT, SLTI, SRA, SRAI) require intermediate signals.

## Lab: Complete ALU

Assign **\$result** for other instrs

```

ANDI $src1_value & $imm;
ORI  $src1_value | $imm;
XORI $src1_value ^ $imm;
ADDI $src1_value + $imm;
SLLI $src1_value << $imm[5:0];
SRLI $src1_value >> $imm[5:0];
AND  $src1_value & $src2_value;
OR   $src1_value | $src2_value;
XOR  $src1_value ^ $src2_value;

SRAI { {32{$src1_value[31]}}, $src1_value } >> $imm[4:0];
SLT  ($src1_value[31] == $src2_value[31]) < $sltu_rslt : {31'b0,$src1_value[31]};
SLTI ($src1_value[31] == $imm[31]) < $sltiu_rslt : {31'b0,$src1_value[31]};
SRA  { {32{$src1_value[31]}}, $src1_value } >> $src2_value[4:0];

```

```

ADD  $src1_value + $src2_value;
SUB  $src1_value - $src2_value;
SLL  $src1_value << $src2_value[4:0];
SRL  $src1_value >> $src2_value[4:0];
SLTU $src1_value < $src2_value;
SLTIU $src1_value < $imm;
LUI  { $imm[31:12], 12'b0 };
AUIPC $pc + $imm;
JAL  $pc + 4;
JALR $pc + 4;

```

Need intermediate result signals for these.

Redwood EDA

45

Figure 10: Complete ALU — all instructions and their expressions

### 7.1 Theory

The ALU output **\$result** is a 32-bit multiplexer driven by the decoded instruction flags. Signed comparisons (SLT, SLTI) require comparing sign bits together with the unsigned compare result. Arithmetic right-shift (SRA, SRAI) is implemented by sign-extending the operand before shifting.

### 7.2 TL-Verilog Code

```

1 // -- Intermediate results needed for signed ops -----
2 $sltu_rslt[31:0] = {31'b0, $src1_value < $src2_value};
3 $sltiu_rslt[31:0] = {31'b0, $src1_value < $imm};
4
5 // -- Full ALU -----
6 $result[31:0] =
7     // I-type ALU

```

```

8      $is_addi    ? $src1_value + $imm      :
9      $is_andi    ? $src1_value & $imm      :
10     $is_ori     ? $src1_value | $imm      :
11     $is_xori    ? $src1_value ^ $imm      :
12     $is_slli    ? $src1_value << $imm[5:0] :
13     $is_srli    ? $src1_value >> $imm[5:0] :
14     $is_srai    ? {{32{$src1_value[31]}}}, $src1_value}
15                  >> $imm[4:0]             :
16     $is_slti    ? ($src1_value[31] == $imm[31])
17                  ? $sltiu_rslt            :
18                  : {31'b0, $src1_value[31]} :
19     $is_sltiu   ? $sltiu_rslt              :
20
21     // R-type
22     $is_add     ? $src1_value + $src2_value :
23     $is_sub     ? $src1_value - $src2_value :
24     $is_and     ? $src1_value & $src2_value :
25     $is_or      ? $src1_value | $src2_value :
26     $is_xor     ? $src1_value ^ $src2_value :
27     $is_sll     ? $src1_value << $src2_value[4:0] :
28     $is_srl     ? $src1_value >> $src2_value[4:0] :
29     $is_sra     ? {{32{$src1_value[31]}}}, $src1_value}
30                  >> $src2_value[4:0]       :
31     $is_sltu    ? $sltu_rslt                :
32     $is_slt     ? ($src1_value[31] == $src2_value[31])
33                  ? $sltu_rslt              :
34                  : {31'b0, $src1_value[31]} :
35
36     // U-type / J-type
37     $is_lui     ? {$imm[31:12], 12'b0}      :
38     $is_auiopc  ? $pc + $imm                :
39     $is_jal     ? $pc + 32'd4                :
40     $is_jalr    ? $pc + 32'd4                :
41
42     // Default
43     32'bx;

```



### Lab Objective

Integrate load (LW/LH/LB) and store (SW/SH/SB) instructions. The Data Memory (DMem) is accessed one cycle after the ALU computes the address, requiring a pipeline bypass for the loaded value.

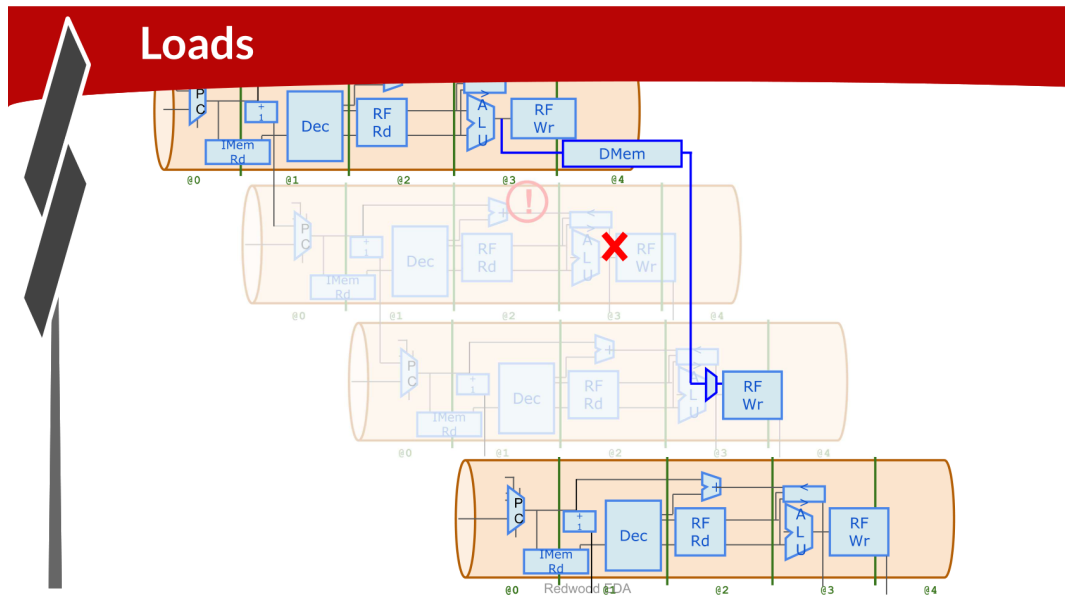


Figure 11: Load instruction pipeline — DMem accessed at stage @4

### 8.1 Theory

- The ALU computes  $\$result = \$src1\_value + \$imm$  as the *memory address*.
- **Stores** write  $\$src2\_value$  to DMem.
- **Loads** read from DMem; the data is available one cycle *after* the ALU stage, so  $\$rf\_wr\_data$  is driven by  $\gg \$ld\_data$  for load instructions.
- An additional hazard must be checked: if the instruction in the previous cycle was a load and its destination matches the current source, a stall may be needed (or the design accepts the latency by using the forwarding already present).

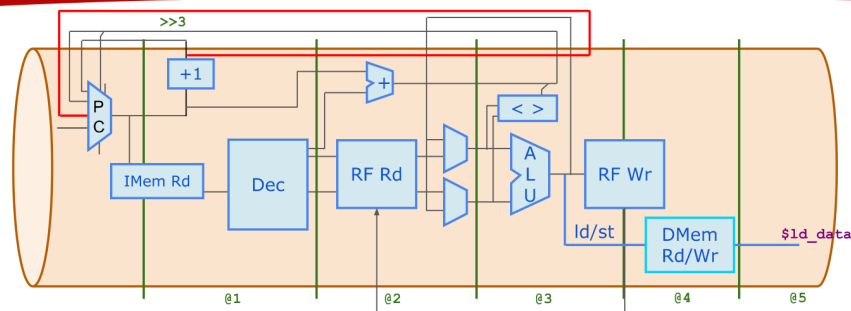
## 8.2 TL-Verilog Code

```

1 // -- Data Memory -----
2 @4
3 // Memory write (store)
4 $dmem_wr_en    = $valid && $is_s_instr;
5 $dmem_wr_data[31:0] = $src2_value;
6 $dmem_addr[3:0]    = $result[5:2];    // word-aligned
7
8 // Memory read (load)
9 $dmem_rd_en    = $valid && $is_load;
10
11 // -- Load data forwarding -----
12 @5
13 $ld_data[31:0] = $dmem_rd_data;
14
15 // -- RF write-back mux -----
16 $rf_wr_en      = ($is_load) ? >>1$valid : $valid && $rd_valid
17                                     && ($rd != 5'b0);
18 $rf_wr_data[31:0] = ($is_load) ? >>1$ld_data : $result;
19 $rf_wr_index[4:0] = ($is_load) ? >>1$rd      : $rd;

```

## Lab: Redirect Loads



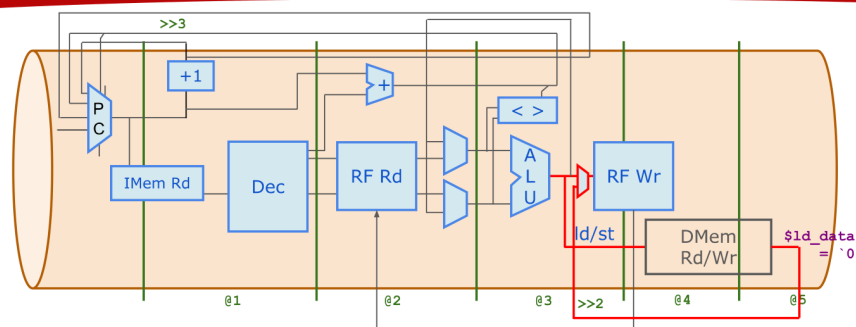
1. Clear `$valid` in the “shadow” of a load (like branch).
2. Select `$inc_pc` from 3 instructions ago for load redirect.
3. Debug. Confirm save.

Figure 12: Load/Store lab instructions

### Lab Objective

Implement unconditional jump instructions JAL and JALR. Both save the return address (PC + 4) in the destination register and redirect the PC.

### Lab: Load Data



1. For loads/stores (*\$is\_load/\$is\_s\_instr*), compute same result as for addi.
2. Add the RF-wr-data MUX to select  $\gg 2 \$ld\_data$  and  $\gg 2 \$rd$  as RF inputs for ! *\$valid* instructions.
3. Enable write of *\$ld\_data* 2 instructions after valid *\$load*.
4. Confirm save.

Redwood EDA

49

Figure 13: Jumps — JAL and JALR implementation slide

### 9.1 Theory

- **JAL** redirects to PC + imm (PC-relative).
- **JALR** redirects to (src1 + imm) & ~1 (register-relative, LSB cleared).
- Both write PC + 4 to *\$rd*.
- The redirect logic is added to the existing PC mux (highest priority after reset).

### 9.2 TL-Verilog Code

```

1 // -- Jump targets -----
2 $jal_tgt_pc[31:0] = $pc + $imm;
3 $jalr_tgt_pc[31:0] = ($src1_value + $imm) & ~32'b1;
4
5 // -- Updated PC mux -----

```

```
6 $valid_jump = $valid && ($is_jal || $is_jalr);
7
8 $pc[31:0] =
9     >>1$reset          ? 32'b0          :
10    >>3$valid_taken_branch ? >>3$br_tgt_pc :
11    >>3$valid_jump && >>3$is_jal ? >>3$jal_tgt_pc :
12    >>3$valid_jump && >>3$is_jalr ? >>3$jalr_tgt_pc :
13                                >>1$pc + 32'd4;
```

## Final Integration

Assemble all components into the complete 3-stage pipelined RISC-V CPU and run the provided test program to verify correctness.

## Lab: Load Data

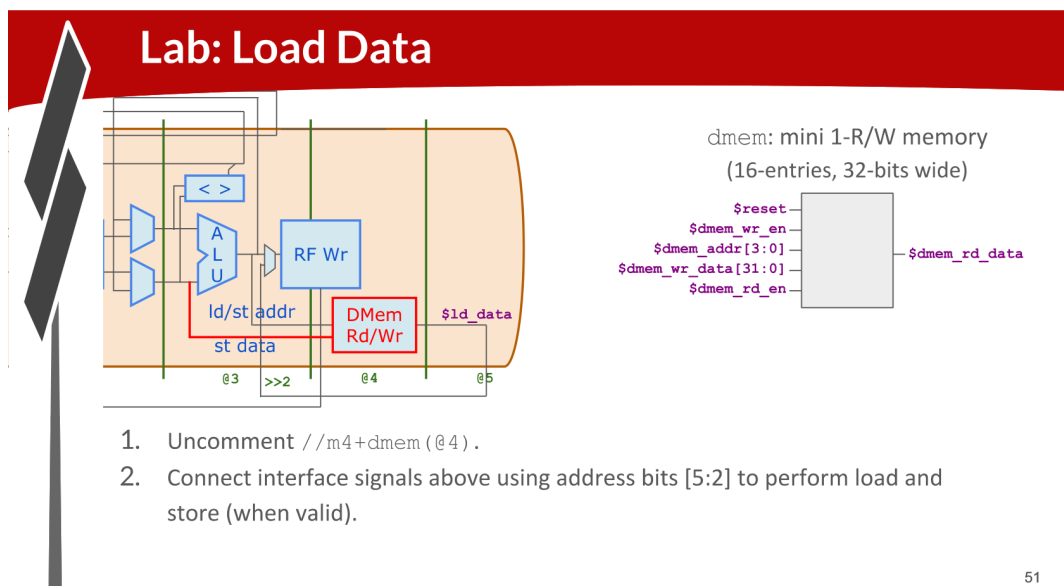


Figure 14: Complete pipelined CPU block diagram

## Lab: Load/Store in Program

Modify the test program to store the final result value to byte address 16, then load it into x17.

```

m4_asm(SW, r0, r10, 10000)
m4_asm(LW, r17, r0, 10000)
```

Update passing condition to look in `xreg[17]`.

Debug. Does the loop properly fall-through and execute store/load.

52

Redwood EDA

Figure 15: Test program and expected output waveform

### 10.1 Complete TL-Verilog Source

The listing below shows the *skeleton structure* of the final Day 5 Makerchip CPU. All signals developed in the individual labs are integrated here.

```

1 \TLV
2 // =====
3 // RISC-V MYTH Workshop --- Day 5 Final CPU
4 // 3-Stage Pipelined RV32I Core (TL-Verilog)
5 // =====
6
7 \m4_TLV_version 1d: tl-x.org
8 \SV
9   'include "mythcore_test.tlv"
10   m4_makerchip_module // simulation module
11
12 \TLV
13 // -- Instruction Memory -----
14 $imem_rd_addr[31:0] = $pc[31:2];
15 $imem_rd_en        = !$reset;
16
17 // -- Pipeline Stage Annotations -----
18 @0
19   $pc[31:0] =
20     >>1$reset          ? 32'b0          :
21     >>3$valid_taken_branch ? >>3$br_tgt_pc :
22     >>3$valid_jump && >>3$is_jal ? >>3$jal_tgt_pc :
23     >>3$valid_jump && >>3$is_jalr ? >>3$jalr_tgt_pc :
24     >>1$pc + 32'd4;
25
26 @1
```

```

27 // -- Fetch -----
28 $instr[31:0] = $imem_rd_data;
29 $start      = >>1$reset && !$reset;
30 $valid      = $reset ? 1'b0
31              : ($start ? 1'b1
32                  : >>3$valid);
33
34 // -- Instruction type decode -----
35 $is_r_instr = $instr[6:2] ==? 5'b01011 ||
36              $instr[6:2] ==? 5'b011x0 ||
37              $instr[6:2] ==? 5'b10100;
38 $is_i_instr = $instr[6:2] ==? 5'b0000x ||
39              $instr[6:2] ==? 5'b001x0 ||
40              $instr[6:2] ==? 5'b11001;
41 $is_s_instr = $instr[6:2] ==? 5'b0100x;
42 $is_b_instr = $instr[6:2] ==? 5'b11000;
43 $is_u_instr = $instr[6:2] ==? 5'b0x101;
44 $is_j_instr = $instr[6:2] ==? 5'b11011;
45
46 // -- Immediate decode -----
47 $imm[31:0] =
48     $is_i_instr ? {{21{$instr[31]}}, $instr[30:20]} :
49     $is_s_instr ? {{21{$instr[31]}}, $instr[30:25],
50                   $instr[11:7]} :
51     $is_b_instr ? {{20{$instr[31]}}, $instr[7],
52                   $instr[30:25], $instr[11:8], 1'b0} :
53     $is_u_instr ? {$instr[31:12], 12'b0} :
54     $is_j_instr ? {{12{$instr[31]}}, $instr[19:12],
55                   $instr[20], $instr[30:21], 1'b0} :
56     32'bx;
57
58 // Register indices
59 $rd_valid  = $is_r_instr || $is_i_instr ||
60              $is_u_instr || $is_j_instr;
61 $rs1_valid = $is_r_instr || $is_i_instr ||
62              $is_s_instr || $is_b_instr;
63 $rs2_valid = $is_r_instr || $is_s_instr ||
64              $is_b_instr;
65 $rd[4:0]   = $instr[11:7];
66 $rs1[4:0]  = $instr[19:15];
67 $rs2[4:0]  = $instr[24:20];
68 $funct3[2:0] = $instr[14:12];
69 $funct7[6:0] = $instr[31:25];
70 $opcode[6:0] = $instr[6:0];
71
72 @2
73 // -- Decode -----
74 $dec_bits[10:0] = {$funct7[5], $funct3, $opcode};
75
76 $is_beq = $dec_bits ==? 11'bx_000_1100011;
77 $is_bne = $dec_bits ==? 11'bx_001_1100011;
78 $is_blt = $dec_bits ==? 11'bx_100_1100011;
79 $is_bge = $dec_bits ==? 11'bx_101_1100011;
80 $is_bltu = $dec_bits ==? 11'bx_110_1100011;
81 $is_bgeu = $dec_bits ==? 11'bx_111_1100011;
82 $is_load = $opcode == 7'b0000011;
83 $is_sw = $dec_bits ==? 11'bx_010_0100011;
84 $is_addi = $dec_bits ==? 11'bx_000_0010011;

```

```

85     $is_add  = $dec_bits ==? 11'b0_000_0110011;
86     $is_jal  = $opcode == 7'b1101111;
87     $is_jalr = $dec_bits ==? 11'bx_000_1100111;
88     // ... (all other decodes as in Lab 4)
89
90     // -- RF Read with bypass -----
91     $rf_rd_en1      = $rs1_valid;
92     $rf_rd_index1[4:0] = $rs1;
93     $rf_rd_en2      = $rs2_valid;
94     $rf_rd_index2[4:0] = $rs2;
95
96     $src1_value[31:0] =
97         (>>1$rf_wr_en && (>>1$rd == $rs1)) ? >>1$result :
98         (>>2$rf_wr_en && (>>2$rd == $rs1)) ? >>2$result :
99                                         $rf_rd_data1;
100    $src2_value[31:0] =
101        (>>1$rf_wr_en && (>>1$rd == $rs2)) ? >>1$result :
102        (>>2$rf_wr_en && (>>2$rd == $rs2)) ? >>2$result :
103                                         $rf_rd_data2;
104
105    @3
106    // -- ALU (as in Lab 5) -----
107    $sltu_rslt[31:0] = {31'b0, $src1_value < $src2_value};
108    $sltiu_rslt[31:0] = {31'b0, $src1_value < $imm};
109
110    $result[31:0] =
111        $is_addi ? $src1_value + $imm      :
112        $is_add  ? $src1_value + $src2_value :
113        $is_or   ? $src1_value | $src2_value :
114        $is_and  ? $src1_value & $src2_value :
115        $is_xor  ? $src1_value ^ $src2_value :
116        $is_sub  ? $src1_value - $src2_value :
117        $is_sll  ? $src1_value << $src2_value[4:0] :
118        $is_srl  ? $src1_value >> $src2_value[4:0] :
119        $is_sltu ? $sltu_rslt              :
120        $is_jal  || $is_jalr ? $pc + 32'd4   :
121        $is_lui  ? {$imm[31:12], 12'b0}      :
122        $is_auipc ? $pc + $imm               :
123        // ... (remaining as in Lab 5)
124        32'bx;
125
126    // -- Branch -----
127    $br_tgt_pc[31:0] = $pc + $imm;
128    $taken_branch =
129        $is_beq ? ($src1_value == $src2_value)      :
130        $is_bne ? ($src1_value != $src2_value)      :
131        $is_blt ? (($src1_value < $src2_value) ^
132                  ($src1_value[31] != $src2_value[31])) :
133        $is_bge ? (($src1_value >= $src2_value) ^
134                  ($src1_value[31] != $src2_value[31])) :
135        $is_bltu ? ($src1_value < $src2_value)      :
136        $is_bgeu ? ($src1_value >= $src2_value)      :
137        1'b0;
138    $valid_taken_branch = $valid && $taken_branch;
139
140    // -- RF Write-back -----
141    $rf_wr_en = $rd_valid && $valid && ($rd != 5'b0)
142                && !$is_load;

```



```
143     $rf_wr_index[4:0] = $rd;
144     $rf_wr_data[31:0] = $result;
145
146     @4
147     // -- Data Memory -----
148     $dmem_wr_en      = $valid && $is_sw;
149     $dmem_wr_data[31:0] = $src2_value;
150     $dmem_addr[3:0]   = $result[5:2];
151     $dmem_rd_en      = $valid && $is_load;
152
153     @5
154     // -- Load write-back -----
155     $rf_wr_en      = >>1$is_load && >>1$valid;
156     $rf_wr_index[4:0] = >>1$rd;
157     $rf_wr_data[31:0] = $dmem_rd_data;
158
159 // Macro: connect to test harness
160 'BOGUS_USE($is_beq $is_bne)
161 m4+cpu_viz(@4)
162
163 \SV_plus
164     always @(posedge clk) begin
165         *passed = |cpu/xreg[10]>>5$value == (1+2+3+...+9);
166     end
```



The simulation passes when register `x10` holds the value **45** (sum of integers 1 through 9) after the test program completes. The Makerchip VIZ panel shows each instruction flowing through the pipeline stages with correct register updates, confirming the design is fully functional.



Figure 18: "YOU DID IT!" — Day 5 completion slide

| Lab   | Topic                        | Key Concept                                   |
|-------|------------------------------|-----------------------------------------------|
| Lab 1 | 3-Cycle <code>\$valid</code> | Pipeline bubble control                       |
| Lab 2 | RF Hazard Bypass             | Read-after-write forwarding                   |
| Lab 3 | Branches                     | PC redirect + squash                          |
| Lab 4 | Complete Instruction Decode  | Full RV32I decode using <code>dec_bits</code> |
| Lab 5 | Complete ALU                 | All RV32I arithmetic/logic ops                |
| Lab 6 | Loads & Data Memory          | DMem, load forwarding                         |
| Lab 7 | Jumps (JAL/JALR)             | Register-relative jump                        |
| Final | Complete Pipelined CPU       | Integration + simulation pass                 |

Day 5 successfully transforms the single-cycle RISC-V core from Day 4 into a **3-stage pipelined** design implementing the full RV32I base instruction set. TL-Verilog's timing abstraction (`»N`) makes pipeline retiming natural and concise, while `$valid`-based squashing cleanly handles both control hazards (branches/jumps) and start-up bubbles. Data hazards are resolved by forwarding (`$src1/2_value` bypass logic), and load-use hazards are handled by the extra write-back stage.

## Workshop Complete — RISC-V MYTH Day 5